RMC_RAW_Redirect_Spec

---

1. Purpose & Block Positioning

RAW Bypass Manager is the Read-After-Write hazard resolution block within the MC Core. Its job is to intercept every incoming read transaction before it reaches the Scheduler and determine whether a pending write in the WR_TCAM already holds the data that read needs. If it does, the data is served directly from the Write Data Buffer — no DRAM round-trip required.

This is a correctness block first, performance block second. Serving stale data from DRAM when a newer write is pending would be a silent data hazard — the kind that passes RTL simulation and fails only in directed hazard tests.

1.1 Position in the Pipeline

The block sits between two well-defined handoff points:

- **Input:** A read transaction that has just been allocated into the RD Status Reg. At this point it has a valid `axi_id`, translated address `{BG, bank, row, col}`, `age` (allocation timestamp), and `resp_slot` reserved in the Resp FIFO.
- **Output A (bypass path):** Covered bytes are routed directly into the Hold-and-Forward buffer, then emitted into the Resp FIFO — bypassing the Scheduler and DRAM entirely. Overshoot bytes are dropped.
- **Output B (normal path):** No valid hit found — the read entry proceeds normally to Scheduler Stage 1.
- **Output C (stall path):** Partial overlap with gaps — the read is held in the RD Status Reg with a STALL status flag. It is re-run through RAW Bypass Manager each cycle until the conflicting write entry retires.

1.2 Read-Only Access

RAW Bypass Manager has read-only access to:

- **WR_TCAM** — all N_WR_ENTRIES valid entries (64 in v1, 96 in v2), specifically: `{valid, BG, bank, row, col, axi_id, entry_idx, data_buf_idx, age}`
- **WR Status Reg** — `valid` and `age` fields per entry, owned by Write Watermark Manager
- **Write Data Buffer** — mask and data fields, indexed via `data_buf_idx` from the matched write entry

RULE: RAW Bypass Manager never modifies any write entry. The matched write proceeds through its full lifecycle (PENDING → ISSUED → DONE → WR_ACK) completely unaffected by a RAW hit.

---

2. Stage A — WR_TCAM Search (Address Exact-Match)

Stage A performs a parallel exact-match search across all N_WR_ENTRIES of the WR_TCAM. It runs every cycle for any read that is either newly allocated or currently stalled.

2.1 Match Key and Valid Gating

The match key is the full translated address of the incoming read:

Field	Width	Source
BG	3b	Address Translator output, stored in RD Status Reg
bank	2b	Address Translator output, stored in RD Status Reg
row	ROW_BITS	Address Translator output, stored in RD Status Reg
col	COL_BITS	Address Translator output, stored in RD Status Reg

Total match key width: BG_BITS + BANK_BITS + ROW_BITS + COL_BITS

A WR_TCAM entry participates in the match only if ALL of the following are true:

1. `wr_status_reg[i].valid == 1`
2. `wr_status[i] ∈ {PENDING, ISSUED}` — not DONE or ERROR
3. `wr_status_reg[i].age <= rd_age` — **the write arrived same cycle or earlier than the read**

Condition 3 is the hit valid condition. A write that was allocated after the read arrived is not a valid RAW source — serving its data would associate a future write with an earlier read, which is incorrect. These entries are filtered out before the match vector is formed.

NOTE: Column is included in the match key. Two writes to the same row but different columns do not alias — they cover different 64B regions. Stage A correctly treats them as independent entries.

2.2 Output: wr_hit_vector

```
wr_hit_vector[N_WR_ENTRIES-1:0]
```

Each bit `i` is set if WR_TCAM entry `i` passes all three valid gating conditions above AND its `{BG, bank, row, col}` exactly equals the incoming read's translated address. Zero or more bits may be set simultaneously.

2.3 Implementation Primitive

Stage A uses XOR-based binary CAM cells — not subtractor-based magnitude comparators. This distinction is critical:

Primitive	Used where	How it works
XOR CAM cell	Stage A (RAW Bypass Manager)	XOR all bits of key vs. stored value; NOR the result. Zero XOR = exact match. Single gate delay.
Subtractor comparator	Scheduler Stage 2 timing gates	`gc - next_legal_cycle`, sign-bit check on bit[GC_WIDTH-1]. Not a CAM.

These must not be conflated in the microarchitecture diagram or RTL. Stage A is a true CAM; Scheduler Stage 2 is a registered `can_*` flag array — no subtractor in the critical path.

2.4 Latency — Open Question

OPEN (OQ-1): Does Stage A add a pipeline register (adds one cycle to every read, including misses) or is it fully combinational and parallel with RD Status Reg allocation? Affects miss-path latency and stall exit timing. Must be resolved before RTL. Recommendation: combinational parallel with allocation to avoid adding latency to the common case (no hazard).

---

3. Stage B — Mask Coverage Check

Stage B runs only when `wr_hit_vector` is non-zero. It selects the winning write entry and determines how much of the read's required data is available in the Write Data Buffer.

3.1 Multi-Hit Tiebreak (Newest Write Wins)

If `wr_hit_vector` has more than one bit set, Stage B selects exactly one write entry. Rule: **newest write wins** — `argmax(wr_status_reg[i].age)` for `i` in hit set.

- Implemented as a `log2(N_WR_ENTRIES)`-deep comparator tree on `age` values from WR Status Reg.
- Rationale: the newest write is the most up-to-date value for that address.
- Lowest-index-wins is explicitly wrong and must not be implemented.

NOTE: Duplicate-address entries (same `{BG,bank,row,col}`) should not arise under correct operation. If they do, newest-wins is the safe tiebreak.

3.2 Mask Coverage Equation

Once the winning write entry `w` is selected:

```
coverable = (rd_mask & wr_mask[w]) == rd_mask
```

Signal	Width	Meaning
`rd_mask`	64b	Byte-level mask of bytes the read requires. Full BL16 non-masked read = all-1.
`wr_mask[w]`	64b	`mask[63:0]` from Write Data Buffer entry at `data_buf_idx` of winning write entry `w`.
`coverable`	1b	1 = write covers all bytes the read needs. 0 = at least one needed byte is missing.

---

4. Hit Classification and Resulting Actions

Four mutually exclusive cases, checked in this order:

4.1 Case 1 — No Valid Hit (wr_hit_vector == 0)

No active write entry with `age <= rd_age` covers this read's address. The read proceeds to Scheduler Stage 1 normally. No intervention.

This also covers the "late write" case — a write exists at the address but arrived after the read. That write is invisible to RAW by construction (filtered in Stage A condition 3).

4.2 Case 2 — Full Hit (wr_mask[w] == all-1)

The winning write covers the entire 64B region. Data is served fully from the Write Data Buffer.

Actions:

- Read full 512b payload from `Write Data Buffer[data_buf_idx]`
- Assemble response: `resp_type=RD_DATA, axi_id=rd_axi_id, data=payload, status=OK`
- Route into Hold-and-Forward buffer (see §5)
- Mark RD Status Reg entry `status = DONE`
- Write entry is NOT touched

4.3 Case 3 — Overlap Hit, Coverable (wr_hit_vector != 0, coverable == 1, wr_mask != all-1)

The winning write covers all bytes the read requires, but the write itself is a partial/masked write. The write's mask is wider than what the read needs — some bytes in the write buffer are "overshoot" relative to what the read asked for.

Actions:

- Extract only the bytes covered by `rd_mask` from the Write Data Buffer payload
- **DROP overshoot bytes** — bytes present in `wr_mask[w]` but not in `rd_mask` are not forwarded
- Assemble response with the covered bytes only
- Route into Hold-and-Forward buffer
- Mark RD Status Reg entry `status = DONE`
- Write entry is NOT touched

NOTE: Overshoot bytes are dropped, not forwarded. The read only gets what it asked for. This is correct — the AXI master issued a specific byte-masked read and must receive exactly those bytes.

4.4 Case 4 — Partial Hit With Gaps (wr_hit_vector != 0, coverable == 0)

A write exists at the address (with `age <= rd_age`) but does not cover all bytes the read requires. Some of the read's required bytes are absent from the Write Data Buffer.

Fetching from DRAM here is incorrect — DRAM still holds the pre-write value for the bytes the write owns. Serving those bytes from DRAM would give the read stale data for the covered bytes.

Action: **STALL**

- Set RD Status Reg entry `status = STALL`
- Do not issue a DRAM read. Do not forward any data.
- Every cycle, re-run Stage A + Stage B for this stalled entry
- Exit condition: the conflicting write entry reaches `status = DONE` (write has been committed to DRAM). At that point, Stage A filters it out (condition 2 fails). Re-check: if `wr_hit_vector == 0` → Case 1, proceed to scheduler. If a different write (also with `age <= rd_age`) now gives `coverable == 1` → Case 3, bypass.
- A write that arrived **after** this read stalled is not a valid exit — it is filtered by condition 3 in Stage A. The stall exits only on retirement of a write that was already a hazard at the time the read arrived.

RULE: A stalled read must NOT degrade to a DRAM fetch. That would silently serve stale data for the overlapping bytes. The only safe exit from stall is: conflicting write retires → re-check.

---

5. Hold-and-Forward Buffer

There is exactly one write port into the Resp FIFO. Two independent response sources can fire in the same cycle: a RAW bypass hit and a DRAM read return. Without buffering, one would be dropped. The Hold-and-Forward buffer is a 2-deep register file that absorbs this collision.

5.1 Structure

Field	Width	Notes
`hold_slot[0:1]`	2 × full resp packet	Each slot: `resp_type`, `axi_id`, `data[511:0]`, `status`
`hold_valid[1:0]`	2b	Per-slot occupancy bit

5.2 Arbitration and Drain Logic

Each cycle:

- If `hold_valid[0] == 1` and Resp FIFO has a free slot: drain slot 0 → Resp FIFO, clear `hold_valid[0]`
- If `hold_valid[1] == 1` and Resp FIFO has a free slot and slot 0 was not just drained this cycle: drain slot 1 → Resp FIFO, clear `hold_valid[1]`
- At most one drain per cycle (single Resp FIFO write port)

New arrivals write into the first free slot. Write-into and drain-from can happen in the same cycle — a new arrival may land in the slot just vacated.

5.3 Backpressure — RAW Bypass Gate

A RAW bypass (Case 2 or 3) may only fire if a hold slot is free and the Resp FIFO has room:

```
raw_bypass_gate = (hold_valid != 2'b11) && gate_resp_fifo_avail
```

If `raw_bypass_gate == 0`, the read stays at PENDING/STALL and retries next cycle.

5.4 Third-Collision Case — Open Question

OPEN (OQ-2): Can a third response arrive in the same cycle both hold slots are occupied and the Resp FIFO is full? Sub-questions: (1) Is this structurally impossible given `gate_resp_fifo_avail` blocks new DRAM reads from issuing when the FIFO is full? (2) If it can occur, does a 3rd slot suffice or is a credit counter needed? Resolve with a rate-limiting proof or structural argument before RTL.

---

6. Signal Interface

All signals are in the MC Core clock domain. No CDC crossings inside RAW Bypass Manager. All intra-MC interfaces are valid-credit (no combinational ready).

6.1 Inputs

Signal	Width	Source	Description
`rd_alloc_valid`	1b	Read WM Manager	Pulsed high whe is allocated into
`rd_bg[BG_BITS-1:0]`	BG_BITS	RD Status Reg	Bank group of ir
`rd_bank[BANK_BITS-1:0]`	BANK_BITS	RD Status Reg	Bank of incomin
`rd_row[ROW_BITS-1:0]`	ROW_BITS	RD Status Reg	Row of incoming
`rd_col[COL_BITS-1:0]`	COL_BITS	RD Status Reg	Column of incor
`rd_axi_id`	AXI_ID_WIDTH	RD Status Reg	AXI ID of the rea response routin
`rd_age[GC_WIDTH-1:0]`	GC_WIDTH	RD Status Reg	Allocation times (from global cyc
`rd_mask[63:0]`	64b	RD Status Reg	Byte mask of by requires
`rd_resp_slot[$clog2(N_RD)-1:0]`	$clog2(N_RD)	RD Status Reg	Pre-reserved Re
`wr_tcam_valid[N_WR-1:0]`	N_WR	WR Status Reg	Per-entry valid b match output)
`wr_tcam_status[N_WR-1:0][1:0]`	N_WR×2	WR Status Reg	Per-entry status (PENDING/ISSU
`wr_tcam_age[N_WR-1:0][GC_WIDTH]`	N_WR×GC_WIDTH	WR Status Reg	Per-entry alloca
`wr_tcam_bg[N_WR-1:0][BG_BITS]`	N_WR×BG_BITS	WR_TCAM	Per-entry bank g
`wr_tcam_bank[N_WR-1:0][BB]`	N_WR×BANK_BITS	WR_TCAM	Per-entry bank

Signal	Width	Source	Description
`wr_tcam_row[N_WR-1:0][RB]`	N_WR×ROW_BITS	WR_TCAM	Per-entry row
`wr_tcam_col[N_WR-1:0][CB]`	N_WR×COL_BITS	WR_TCAM	Per-entry colum
`wr_tcam_data_idx[N_WR-1:0][4:0]`	N_WR×5	WR_TCAM	Per-entry Write
`wdb_mask[N_WR-1:0][63:0]`	N_WR×64	Write Data Buffer	Byte mask per e
`wdb_data[N_WR-1:0][511:0]`	N_WR×512	Write Data Buffer	Full 512b payloa only)
`hold_valid[1:0]`	2b	Hold-and-Forward	Occupancy of h
`gate_resp_fifo_avail`	1b	Resp FIFO	1 = Resp FIFO unreserved slot
`stall_rd_re_check`	1b/entry	RD Status Reg	Asserted each c currently stalled

6.2 Outputs

Signal	Width	Destination	Description
`raw_hit`	1b	RD Status Reg	1 = a bypass or stall decision was made this cycle
`raw_bypass_en`	1b	Hold-and-Forward	1 = valid bypass data ready to write into a hold slot
`raw_bypass_data[511:0]`	512b	Hold-and-Forward	Masked payload (covered bytes only, overshoot zeroed)
`raw_bypass_mask[63:0]`	64b	Hold-and-Forward	rd_mask — indicates which bytes in raw_bypass_data are valid
`raw_bypass_axi_id`	AXI_ID_WIDTH	Hold-and-Forward	AXI ID from the **read** entry (not the write) — for response routing
`raw_bypass_slot`	$clog2(N_RD)	Hold-and-Forward	Resp FIFO slot to target
`raw_stall_en`	1b	RD Status Reg	1 = set this read entry status to STALL
`raw_no_hazard`	1b	Scheduler Stage 1	1 = no valid RAW conflict, read may proceed to

Signal	Width	Destination	Description
			TCAM-based scheduler
`wr_hit_vector[N_WR-1:0]`	N_WR	Internal (Stage B)	CAM hit vector — internal signal, not exported outside the block

NOTE on `raw_bypass_axi_id`: The response must carry the **read's** AXI ID (`rd_axi_id`), not the matched write's `axi_id`. The write and read may come from different AXI masters. Using the write's ID here would route the response to the wrong master. This is the masking your teammate referred to — the write's AWID is masked out and replaced with the read's ARID in the response path.

---

7. Operational Flow — Cycle by Cycle

7.1 New Read Allocation — No Hazard

- **Cycle N:** Read allocated into RD Status Reg. `rd_alloc_valid` asserted.
- **Cycle N (or N+1, see OQ-1):** Stage A evaluates `wr_hit_vector`. All bits zero (or all filtered by age/status gate).
- `raw_no_hazard` asserted. Read entry status remains PENDING.
- **Cycle N+1 (or N+2):** Read entry enters Scheduler Stage 1 normally.

7.2 New Read Allocation — Full RAW Hit

- **Cycle N:** `rd_alloc_valid` asserted. Stage A fires, `wr_hit_vector` non-zero.
- Stage B: if multi-hit, `argmax(age)` selects winner. `coverable` evaluated — `wr_mask == all-1`.
- `raw_bypass_gate` checked. If clear: full 512b payload latched from Write Data Buffer.
- Response assembled with `axi_id = rd_axi_id` (not the write's ID).
- Hold-and-Forward: bypass packet written into first free hold slot.
- RD Status Reg entry `status = DONE`.
- Write entry untouched — continues to DRAM normally.

7.3 New Read Allocation — Overlap Hit, Coverable

- **Cycle N:** Stage A hits, `coverable == 1`, `wr_mask != all-1`.
- Covered bytes extracted from Write Data Buffer payload using `rd_mask`.
- Overshoot bytes (in `wr_mask` but not `rd_mask`) are **dropped** — zeroed in the response payload.
- `raw_bypass_en` asserted (subject to `raw_bypass_gate`). Response carries `rd_mask` to indicate valid byte lanes.

- RD Status Reg entry `status = DONE`. Write entry untouched.

7.4 New Read Allocation — Partial Hit With Gaps

- **Cycle N:** Stage A hits, `coverable == 0`.
- `raw_stall_en` asserted. RD Status Reg entry `status = STALL`.
- Read does NOT proceed to Scheduler.
- **Every subsequent cycle:** Stage A + Stage B re-evaluated for this stalled entry.
- Exit: conflicting write reaches `status = DONE` → filtered from Stage A → `wr_hit_vector` may go to zero → `raw_no_hazard` → proceed to scheduler.
- A write allocated **after** this read does not form a valid exit — it is filtered by the `wr_age <= rd_age` condition.

7.5 DRAM Return + RAW Bypass Same Cycle

- Both `raw_bypass_en` and a DRAM read return assert same cycle.
- Each writes into one of the two hold slots.
- `hold_valid` becomes `2'b11`.
- Drain logic drains one per cycle on subsequent cycles as Resp FIFO frees up.

---

8. Correctness Invariants

These must hold at all times. Any implementation that violates these is architecturally incorrect regardless of timing closure:

#	**Invariant**
I-1	A write entry matched by a RAW hit is never retired early or modified by RAW Bypass Manager. It proceeds through PENDING → ISSUED → DONE → WR_ACK independently.
I-2	A read with `coverable == 0` must never be issued to DRAM while the conflicting write is still active. Fetching from DRAM returns pre-write data for the covered bytes — silent corruption.
I-3	Newest-write-wins on multi-hit. Lowest-index-wins is incorrect and must not be implemented.
I-4	A RAW bypass may only fire when `raw_bypass_gate == 1`. Firing without a free hold slot drops data silently.
I-5	Stage A only considers write entries with `status ∈ {PENDING, ISSUED}`. A DONE write is no longer a hazard source.
I-6	Column is part of the match key. Writes to the same row but different columns are not hazards to each other.

#	Invariant
I-7	Only writes with `wr_age <= rd_age` are valid RAW sources. A write allocated after the read arrived must not be used to serve that read — it is a future write, not a hazard.
I-8	Response routing always uses the **read's** `axi_id`, never the matched write's `axi_id`. The write and read may originate from different AXI masters.
I-9	On an overlap hit (Case 3), only bytes covered by `rd_mask` are forwarded. Overshoot bytes are dropped. The read receives exactly what it requested.

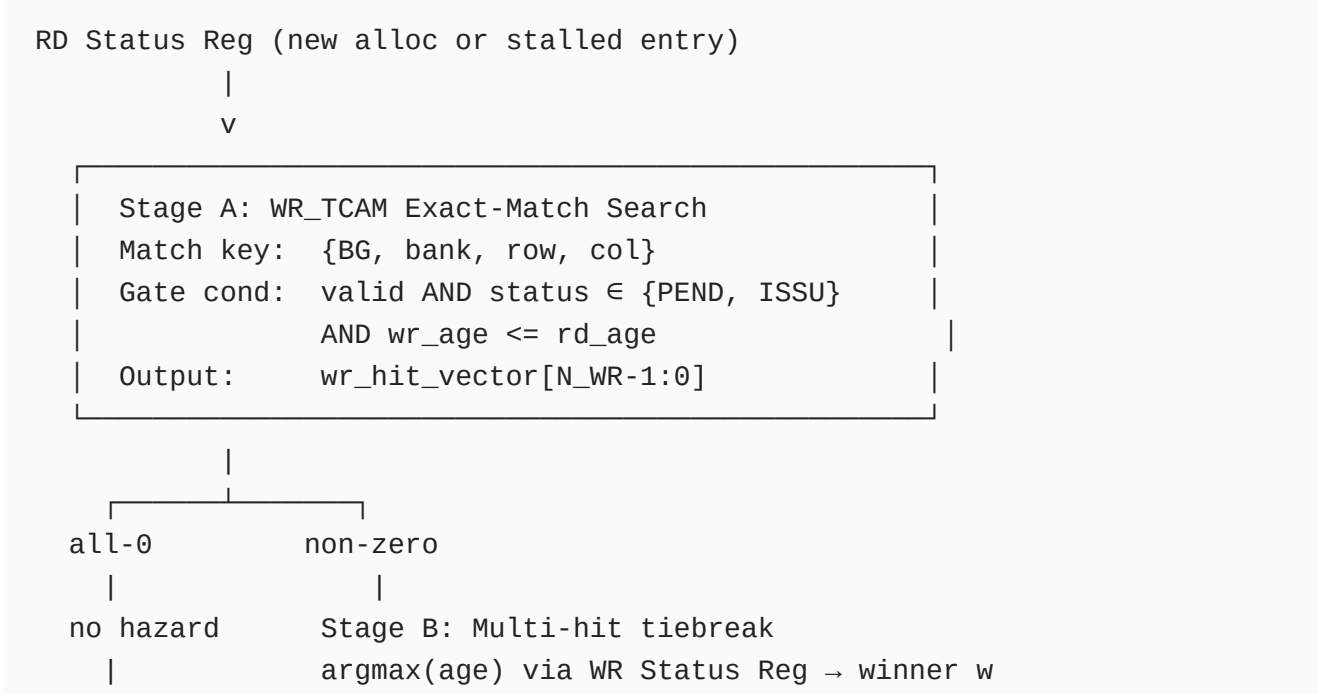
---

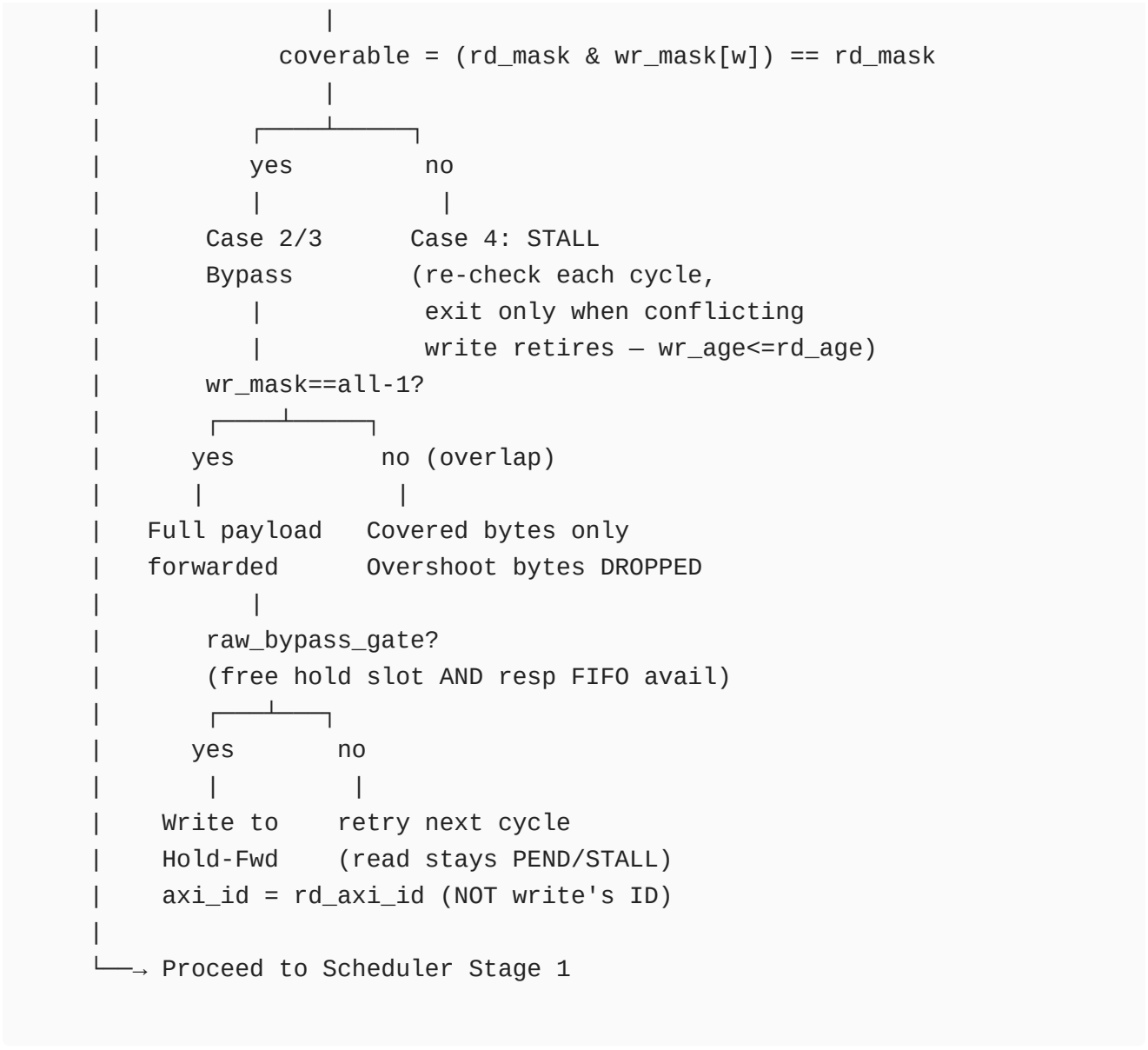
9. Open Questions

#	Topic	Question	Impact
OQ-1	Stage A latency	Combinational parallel with RD Status Reg allocation, or pipelined (adds one cycle to every read)? Affects miss-path latency and stall exit timing.	ROB watermark, miss-path cycle count
OQ-2	3rd Hold-Forward collision	Can a 3rd response arrive when both hold slots are occupied? Is this rate-limited by construction via `gate_resp_fifo_avail`? If not, does a 3rd slot suffice or is a credit counter needed?	Hold-Forward buffer depth, backpressure design

---

10. Block Summary




coverable = (rd_mask & wr_mask[w]) == rd_mask
yes
no
Case 2/3
Bypass
Case 4: STALL
(re-check each cycle,
exit only when conflicting
write retires — wr_age<=rd_age)
wr_mask==all-1?
yes
no (overlap)
Full payload
forwarded
Covered bytes only
Overshoot bytes DROPPED
raw_bypass_gate?
(free hold slot AND resp FIFO avail)
yes
no
Write to
Hold-Fwd
retry next cycle
(read stays PEND/STALL)
axi_id = rd_axi_id (NOT write's ID)
Proceed to Scheduler Stage 1